

**Министерство общего и профессионального образования
Российской Федерации**
Федеральное государственное автономное образовательное учреждение
высшего образования
**«Пермский национальный исследовательский политехнический
университет»**

ОТЧЁТ

по теме:

**АВТОМАТИЗАЦИЯ РАБОЧЕГО МЕСТА ДИСПЕТЧЕРА ПО
РАСПИСАНИЮ**

Выполнил: студент группы РИС-25-2Б
Ильинов Фёдор Михайлович
Принял: Доц. Полякова О. А.

Пермь 2026

СОДЕРЖАНИЕ

Введение	3
1 Постановка задачи и анализ предметной области	4
1.1 Описание предметной области	4
1.2 Функциональные требования	4
2 Используемые инструменты и технологии	5
2.1 Языки программирования и среды разработки	5
2.2 Библиотеки и API	5
2.3 Инструменты развёртывания	5
3 Архитектура приложения	7
3.1 Общая архитектура	7
3.2 Серверная часть (Go)	7
3.3 Desktopный клиент (C++/Qt6)	8
3.4 Структура базы данных	9
4 Реализация ключевых решений	10
4.1 OpenAPI-генерация кода	10
4.2 Импорт расписания из XLSX	10
4.3 Экспорт в iCalendar	11
4.4 Desktopный клиент на Qt6	11
4.5 Докеризация приложения	11
5 Результаты разработки	12
Заключение	13
Список использованных источников	14

ВВЕДЕНИЕ

В современных образовательных учреждениях управление расписанием учебных занятий является одной из ключевых организационных задач. Диспетчеру по расписанию необходимо оперировать большим объёмом данных: учебные группы, преподаватели, аудитории, дисциплины, семестры — и обеспечивать их непротиворечивую связку. Традиционные подходы (Excel-таблицы, бумажные журналы) приводят к ошибкам, дублированию информации и низкой оперативности внесения изменений.

Автоматизированное рабочее место (АРМ) диспетчера по расписанию призвано решить задачи централизованного учёта всех элементов расписания, импорта данных из Excel-файлов, визуализации недельной сетки занятий и экспорта расписания в формат iCalendar для интеграции с календарями Google и Apple.

Целью настоящей работы является разработка АРМ диспетчера по расписанию — клиент-серверного приложения с REST API, десктопным клиентом на C++/Qt6 и веб-интерфейсом. Для достижения поставленной цели необходимо решить следующие задачи:

- проектирование схемы базы данных и REST API на основе OpenAPI 3.0;
- реализация серверной части на Go с автогенерацией кода по OpenAPI-спецификации;
- разработка десктопного клиента на C++/Qt6 с автогенерацией API-клиента;
- реализация импорта расписания из ZIP-архивов с XLSX-файлами;
- реализация экспорта расписания в формат iCalendar;
- контейнеризация приложения средствами Docker.

1 Постановка задачи и анализ предметной области

1.1 Описание предметной области

Предметная область — управление учебным расписанием высшего учебного заведения. Система оперирует следующими сущностями:

- **Подгруппа** (Subgroup) — академическая группа студентов;
- **Преподаватель** (Teacher) — сотрудник кафедры, проводящий занятия;
- **Аудитория** (Location) — место проведения занятия;
- **Дисциплина** (Subject) — учебный предмет;
- **Расписание** (Timetable) — семестровый период с указанием дат и типа недели (числитель/знаменатель);
- **Занятие** (Lesson) — проведение дисциплины в определённое время, с привязкой к преподавателю, аудитории и подгруппам.

1.2 Функциональные требования

На основании анализа предметной области сформулированы следующие функциональные требования:

- CRUD-операции для всех справочных сущностей;
- CRUD-операции для занятий с гибким связыванием преподавателей и аудиторий;
- импорт расписания из ZIP-архива, содержащего XLSX-файлы определённого формата;
- экспорт расписания в формате iCalendar (.ics) для интеграции с календарями;
- визуализация расписания в виде недельной сетки;
- разграничение доступа по ролям.

2 Используемые инструменты и технологии

2.1 Языки программирования и среды разработки

Серверная часть — Go. Выбор обусловлен высокой производительностью, встроенной поддержкой конкурентности, простым синтаксисом и широкими возможностями для создания HTTP-сервисов [1].

Десктопный клиент — C++20 с Qt 6. Выбор Qt 6 обусловлен кроссплатформенностью, богатым набором виджетов и встроенной поддержкой сетевых запросов [2].

2.2 Библиотеки и API

Серверная часть использует следующие основные библиотеки:

- **pgx/v5** — PostgreSQL драйвер с пулом соединений;
- **oapi-codegen** — генерация Go-кода из OpenAPI [3];
- **excelize/v2** — чтение и парсинг XLSX-файлов;
- **golang-ical** — генерация iCalendar;
- **goose/v3** — миграции базы данных [4];
- **sqlc** — генерация type-safe Go-кода из SQL [5].

Десктопный клиент на C++/Qt 6 использует:

- Qt 6 (Core, Gui, Widgets) — построение GUI и сетевое взаимодействие;
- OpenAPI Generator (cpp-qt-client) — генерация C++ Qt-клиента из OpenAPI [6].

2.3 Инструменты развёртывания

- **Docker** — контейнеризация сервисов [7];
- **Docker Compose** — оркестрация многоконтейнерного приложения;
- **PostgreSQL** — реляционная база данных [8];

- **Nginx** — обратный прокси и раздача статических файлов.

3 Архитектура приложения

3.1 Общая архитектура

Приложение построено по клиент-серверной архитектуре с единым API-контрактом. Центральным элементом является OpenAPI 3.0 спецификация [9], описывающая все эндпоинты, типы данных и форматы ответов REST API.

Система состоит из нескольких компонентов:

- а) **Сервер** (Go)—обработка HTTP-запросов, бизнес-логика, работа с базой данных;
- б) **Десктопный клиент** (C++/Qt6)—нативное приложение для диспетчера;
- в) **Веб-интерфейс** (React/TypeScript)—публичное приложение для студентов.

3.2 Серверная часть (Go)

Серверная часть использует слоистую архитектуру:

- **Слой API**—автоматически сгенерированные типы и интерфейс StrictServerHandler (oapi-codegen) [3];
- **Слой приложения** (Application)—реализация бизнес-логики и хендлеров REST-эндпоинтов;
- **Слой репозитория** (Repository)—работа с базой данных через sqlc [5];
- **Слой библиотек** (Lib)—парсер XLSX, сериализатор ICS.

Запуск сервера осуществляется в **main.go**: загрузка конфигурации, подключение к PostgreSQL [8], создание Application, обращение в StrictHandler, запуск HTTP-сервера.

На рисунке 3.1 представлена диаграмма классов десктопного клиента.



- **TimetablePainter**—отрисовка недельной сетки расписания средствами QGraphicsScene/QGraphicsView; агрегирует UiLesson; зависит от OAILocation;

- **LocationsEditorWidget** — виджет редактирования аудиторий; зависит от **OAIDefaultApi**;
- **OAIDefaultApi** — сгенерированный по OpenAPI-спецификации класс для взаимодействия с REST API; наследуется от **OAIObject**;
- **OAILocation** — модель данных локации; наследуется от **OAIObject**.

Аналогичную структуру имеют редакторы **SubgroupEditorWidget**, **TeacherEditorWidget**, **SubjectEditorWidget**, предназначенные для редактирования подгрупп, преподавателей и дисциплин соответственно. Каждый из них зависит от **OAIDefaultApi** и построен по тому же принципу, что и **LocationsEditorWidget**.

3.4 Структура базы данных

База данных реализована средствами PostgreSQL 16 и включает таблицы: **subgroups**, **teachers**, **locations**, **subjects**, **timetables**, **lessons**, **subgroup_assignments**, **teacher_location_assignments**. Миграции выполняются с помощью **goose** [4], генерация type-safe запросов — с помощью **sqlc** [5].

4 Реализация ключевых решений

4.1 OpenAPI-генерация кода

Одним из ключевых достижений является полная генерация серверного кода по OpenAPI-спецификации [3]. Инструмент **oapi-codegen** принимает файл **openapi/openapi.yaml** и создаёт типы данных, интерфейс **StrictServerHandler** и HTTP-маршрутизатор.

Разработчику достаточно реализовать методы заданного интерфейса. Аналогично для десктопного клиента: **OpenAPI Generator (cpp-qt-client)** [6] из той же спецификации генерирует класс **OAIDefaultApi** со всеми методами API.

4.2 Импорт расписания из XLSX

Модуль импорта реализован в пакете **internal/lib/xlsxParser.go**. Класс **Parser** выполняет разбор XLSX-файлов в формате, используемом диспетчерской службой ПНИПУ.

Основные этапы работы:

- а) разархивация ZIP-файла, полученного через multipart-форму;
- б) построчный обход XLSX-таблицы для каждой учебной группы;
- в) парсинг ячейки занятия с помощью регулярных выражений;
- г) формирование staging-данных во временных таблицах в рамках транзакции;
- д) сравнение хешей для избежания дублирования;
- е) перенос в основные таблицы.

Транзакционный подход гарантирует атомарность импорта: при ошибке на любом этапе все изменения откатываются.

4.3 Экспорт в iCalendar

Модуль экспорта реализован в пакете **internal/lib/icsSerializer.go**. Функция **SerializeICS** принимает список занятий и имя владельца расписания и возвращает байтовый массив в формате iCalendar. Экспорт доступен как в десктопном клиенте (через буфер обмена), так и через REST API.

4.4 Десктопный клиент на Qt6

Десктопный клиент использует сгенерированный по OpenAPI-спецификации класс **OAIDefaultApi**. Все сетевые вызовы асинхронны (сигналы Qt), что обеспечивает отзывчивость интерфейса. Каждый редактор сущности содержит поля для отображения и изменения данных, кнопки сохранения и удаления, подключаемые к соответствующим сигналам API.

Визуализация расписания реализована через **TimetablePainter**, отрисовывающий недельную сетку: дни недели по горизонтали, временные слоты по вертикали, занятия в виде цветных блоков. Цвета соответствуют официальному бренд-буку ПНИПУ и типу занятия: лекция, практика, КСР, лабораторное занятие.

4.5 Докеризация приложения

Приложение полностью контейнеризировано. Docker Compose описывает четыре сервиса:

- **postgres** — база данных PostgreSQL 16 [8];
- **backend** — Go-сервер;
- **frontend** — веб-интерфейс (React);
- **nginx** — обратный прокси.

Запуск осуществляется одной командой **docker compose up**.

5 Результаты разработки

В результате работы разработано полнофункциональное клиент-серверное приложение. Приложение предоставляет диспетчеру следующие возможности:

- CRUD-управление справочными данными (группы, преподаватели, аудитории, дисциплины, семестры);
- CRUD-управление занятиями с гибкой привязкой преподавателей и аудиторий;
- импорт расписания из ZIP-архивов с XLSX-файлами формата ПНИПУ;
- экспорт расписания в iCalendar для Google/Apple календарей;
- просмотр расписания в недельной сетке в десктопном приложении;
- разграничение доступа по ролям.

Автор считает наиболее значимыми достижениями работы следующие:

а) **Импорт данных из Excel-файлов** — главное достоинство программы. Обеспечена совместимость с форматом файлов, используемым в ВУЗе. Программа автоматически обрабатывает время, дни недели, подгруппы (в том числе не заданные явно), чётность недели и прочие данные.

б) **OpenAPI-генерация кода** — единая спецификация автоматически генерирует сервер на Go и клиент на C++/Qt, исключая рассинхронизацию между API и клиентами.

в) **Экспорт iCalendar** — после однократной подписки студенту не требуется выполнять дополнительные действия: изменения, вносимые диспетчером, автоматически синхронизируются с календарём пользователя.

г) **Докеризация** — полный стек запускается одной командой **docker compose up**.

Виджеты занятий используют цветовую схему из официального бренд-бука ПНИПУ — цвета соответствуют типу занятия: лекция, практика, КСР, лабораторное занятие.

ЗАКЛЮЧЕНИЕ

В ходе выполнения творческой работы разработано автоматизированное рабочее место диспетчера по расписанию. Поставленная цель достигнута, все задачи решены.

Проведён анализ предметной области, спроектирована структура реляционной базы данных, разработана OpenAPI-спецификация REST API. Реализован сервер на Go с автогенерацией кода по спецификации. Разработан десктопный клиент на C++/Qt6 с автогенерацией API-клиента. Реализован импорт расписания из XLSX-файлов с транзакционным механизмом staging-таблиц и экспорт в формат iCalendar. Выполнена контейнеризация приложения средствами Docker [7].

Проект имеет перспективы развития. В будущем возможно добавление автоматического составления расписания в зависимости от заданных параметров: рабочие дни преподавателей, учебное время групп, учёт санитарно-эпидемиологических правил и норм. Также возможна реализация формирования печатного варианта текущей схемы расписания.

Полученные навыки проектирования архитектуры клиент-серверных приложений, работы с OpenAPI, Go, C++/Qt, реляционными базами данных и Docker имеют практическую ценность и могут быть применены при разработке более сложных программных систем.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Google. Go Documentation. — 2026. — Режим доступа: <https://go.dev/doc> (дата обращения: 20.05.2026).
2. Group Qt. Qt 6 Documentation. — 2026. — Режим доступа: <https://doc.qt.io/qt-6/> (дата обращения: 20.05.2026).
3. DeepMap. oapi-codegen: Go code generator for OpenAPI. — 2026. — Режим доступа: <https://github.com/deepmap/oapi-codegen> (дата обращения: 20.05.2026).
4. Pressly. goose: Database migrations for Go. — 2026. — Режим доступа: <https://github.com/pressly/goose> (дата обращения: 20.05.2026).
5. sqlc. sqlc: Type-safe Go SQL. — 2026. — Режим доступа: <https://sqlc.dev> (дата обращения: 20.05.2026).
6. Generator OpenAPI. OpenAPI Generator: cpp-qt-client. — 2026. — Режим доступа: <https://openapi-generator.tech/docs/generators/cpp-qt-client> (дата обращения: 20.05.2026).
7. Inc. Docker. Docker Documentation. — 2026. — Режим доступа: <https://docs.docker.com> (дата обращения: 20.05.2026).
8. Group PostgreSQL Global Development. PostgreSQL Documentation. — 2026. — Режим доступа: <https://www.postgresql.org/docs/> (дата обращения: 20.05.2026).
9. Initiative OpenAPI. OpenAPI Specification v3.0. — 2026. — Режим доступа: <https://swagger.io/specification/> (дата обращения: 20.05.2026).